Algorithm Engineering

False Sharing, Race Conditions, and Schedules

Mark Blacher

Friedrich Schiller University Jena

Winter Semester of 2026/27

Your Project (Best Case)

- version control with **Git**
- the use of **CMake** to build the project
- correct, efficient, parallel, vectorized, cache-friendly, well organized, **commented**, (portable) C++/C code
- If you write a **library**, the **API** should be **clear** and **well documented**
- If you create a **standalone executable**, **document well** the **usage** and the **available command line options** (command line parameters can be parsed with `getopt`)
- **unit tests** (for example with **Catch2**)

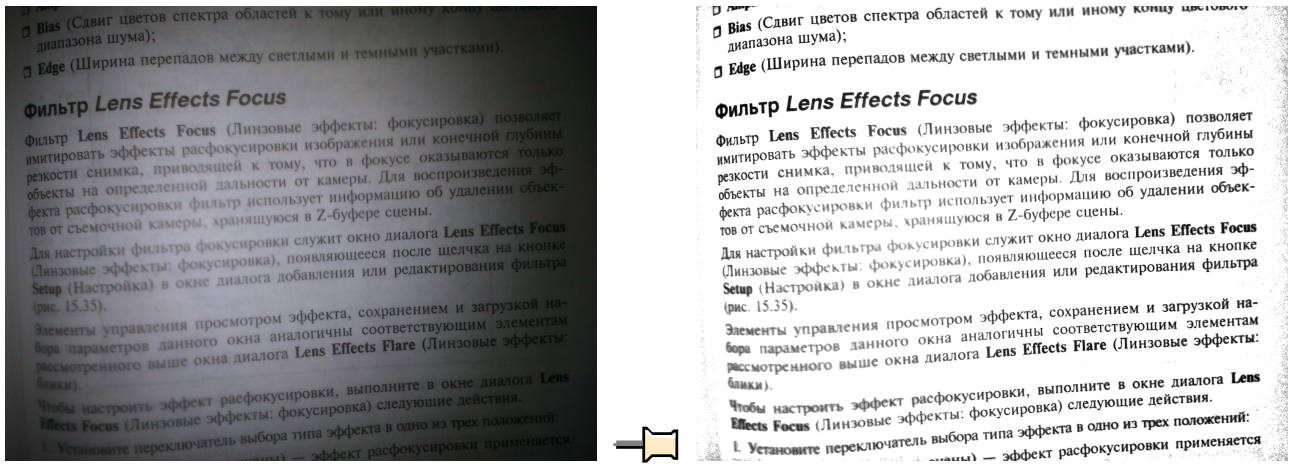
Project 1: **Enhancer** for **Scanned Images**

Input: portable pix map (ppm) color image(s) in ASCII-format

Output: cleaned ppm image(s) in ASCII-format

Motivation: free tool (**executable**) for improving the quality of scanned documents before printing (or sending) them

Keywords: noise reduction, background removal, image filter



For more information see:

[Adaptive Thresholding Methods for Documents Image Binarization]

[Adaptive Thresholding Using the Integral Image]

[Binarization of Historical Document Images Using the Local Maximum and Minimum]

For reading and writing images you can also use [stb]

Project 2: **Generating 2D Datasets** With **Similar Statistical Properties**

Input: a 2D point cloud

Output: a 2D point cloud with same statistical properties (for a given shape)

Motivation: the current approach is slow (written in Python); perhaps it can be done in a more general way than through hordecoded shapes in the program

Keywords: simulated annealing, genetic algorithms, mean, standard deviation, correlation

For more information see:

https://www.autodesk.com/research/publications/same-stats-different-graphs

https://github.com/khuyentran1401/same-stats-different-graphs

Project 3: **Einsum** With **Batch Matrix Multiplication**

Input: two NumPy arrays (tensors) and a format string

Output: NumPy array computed on the basis of the format string

Motivation: the einsum implementation in NumPy is slow and single threaded, but can easily be sped up with a batch matrix multiplication

Keywords: transposition, tensor computations, batch matrix multiplication

For more information see:

https://github.com/jcmgray/einsum_bmm

https://www.tensors.net/p-tutorial-1

Transpose-Transpose-GEMM-Transpose → in our case it is Transpose-Transpose-BMM-Transpose

Your Own Project?

- You can choose one of three mentioned projects
- But, it is also possible to come up with your own project
- If you have an **own project idea**, **describe it to me in an e-mail** (or in the PC-Pool)
- If your own project is suitable for the course, then you can do it

Send me an **e-mail until November 17rd** with a **description of your own project**, **or**, with the **information which of the three presented projects** you would like to do.

It might be beneficial for you to **add me to your repository**.

mark.blacher@uni-jena.de

Avoid False Sharing Among Threads

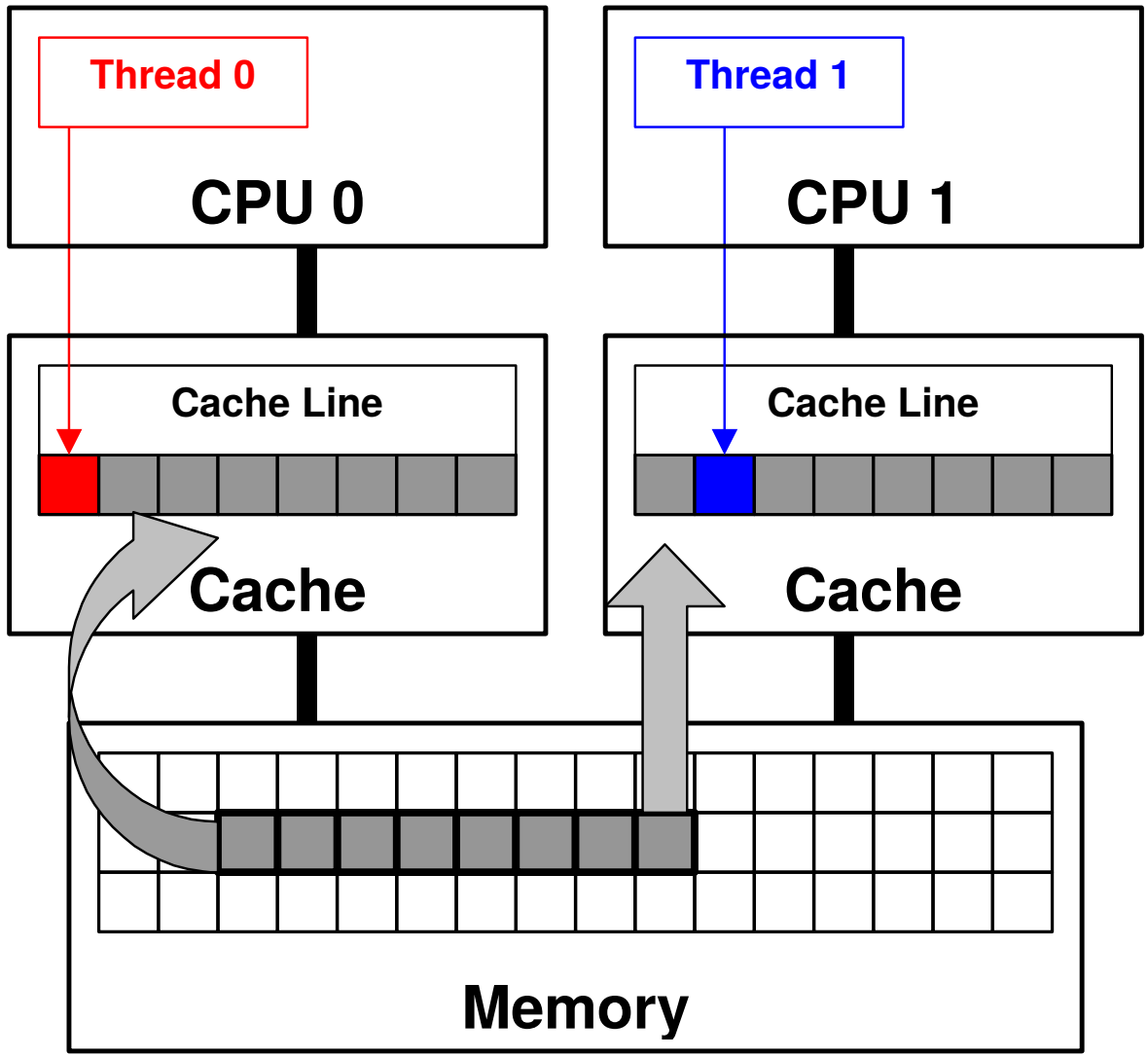
Cache Line

The smallest unit of memory that can be transferred between the main memory and the cache (**typically 64 bytes**).

False Sharing

False sharing occurs when **threads on different processors modify variables that reside on the same cache line**. This invalidates the cache line and forces a memory update to maintain cache coherency.

False Sharing



Click here for more information

Code Example **With False Sharing**



false_sharing.cpp

```cpp
#include <iostream>
#include <omp.h>

using namespace std;

int main() {
  const int threads = 4; // number of threads
  volatile long arr[threads]; // no compiler optimizations on arr
  double start_time = omp_get_wtime(); // wall clock time in seconds
#pragma omp parallel num_threads(threads)
  {
    const int thread_id = omp_get_thread_num();
    for (long i = 0; i < 1000000000; ++i) {
      // each thread uses (probably) the same cache line
      ++arr[thread_id];
    }
  }
  // cout the byte where arr starts in the cache line
  cout <<  ((long) &arr[0]) % 64 << endl;
  cout << omp_get_wtime() - start_time << " seconds\n";
}
```

Code Example **Without False Sharing**

no_false_sharing.cpp

```cpp
#include <iostream>
#include <omp.h>

using namespace std;

int main() {
  const int threads = 4; // number of threads
  const int PAD = 8; // padding value to avoid false sharing
  volatile long arr[threads * PAD]; // no compiler optimizations on arr
  double start_time = omp_get_wtime(); // wall clock time in seconds
#pragma omp parallel num_threads(threads)
  {
    const int thread_id = omp_get_thread_num();
    for (long i = 0; i < 1000000000; ++i) {
      // each thread increments a value on a different cache line
      ++arr[thread_id * PAD];
    }
  }
  cout << omp_get_wtime() - start_time << " seconds\n";
}
```

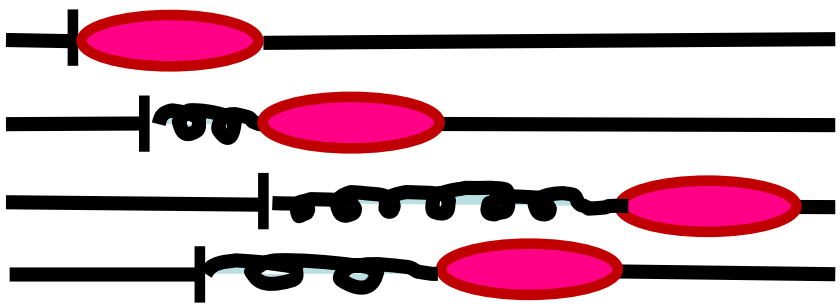
Prevent Race Conditions With Mutual Exclusion

Race Condition

A race condition occurs when **two or more threads try to change shared data** at the same time.

Mutual Exclusion

A form of synchronization in which **only one thread** at a time **can execute a block of code**.



Mutual Exclusion Constructs

- `#pragma omp critical` *// critical block runs on one thread at a time*
- `std::mutex` *// an instance of std::mutex should be shared between threads*
- `#pragma omp atomic` *// updates a memory location atomically*
- ...

Usually a **thread** that reaches a mutual exclusion construct **waits until it gains exclusive access**.

Can You Spot the **Race Condition Bug**?



race_condition.cpp

```
1   double pi = 0.0;
2 #pragma omp parallel num_threads(4)
3   {
4     int num_threads = omp_get_num_threads();
5     int thread_id = omp_get_thread_num();
6     double sum_local = 0.0; // for summing up heights locally
7     for (int i = thread_id; i < num_steps; i += num_threads) {
8       double x = (i + 0.5) * width; // midpoint
9       sum_local = sum_local + (1.0 / (1.0 + x * x)); // add new height
10    }
11    pi += sum_local * 4 * width;
12  }
```

Why Is This **Code Slow**?



pi_slow.cpp

```
1   double pi = 0.0;
2 #pragma omp parallel num_threads(4)
3   {
4     int num_threads = omp_get_num_threads();
5     int thread_id = omp_get_thread_num();
6     for (int i = thread_id; i < num_steps; i += num_threads) {
7       double x = (i + 0.5) * width; // midpoint
8 #pragma omp atomic
9       pi = pi + (4.0 / (1.0 + x * x)) * width;
10    }
11  }
```

The #pragma omp for Construct

#pragma omp for splits up a for loop within a parallel region among the threads in a team

pragma_omp_for.cpp

```cpp
image *denoise_image(image *in) { // mock denoise
  image *out = nullptr;
  for (volatile int i = 0; i < 20000000; ++i) {}
  return out;
}

int main() {
  int amount_images = 64;
  vector<image *> images(amount_images, nullptr); // mock images
  vector<image *> denoised_images(amount_images); // mock output
  double start = omp_get_wtime();
#pragma omp parallel
  {
#pragma omp for // denoise a bunch of images
    for (int i = 0; i < amount_images; ++i) {
      denoised_images[i] = denoise_image(images[i]);
    }
  }
  cout << omp_get_wtime() - start << " seconds" << endl;
}
```

Unbalanced Workload

unbalanced_workload.cpp

```
1  image *denoise_image(image *in, int scale) { // mock denoise
2    image *out = nullptr;
3    const int loop_until = 20000 * scale; // unbalanced workload
4    for (volatile int i = 0; i < loop_until; ++i) {}
5    return out;
6  }
7
8  int main() {
9    int amount_images = 64;
10   vector<image *> images(amount_images, nullptr); // mock images
11   vector<image *> denoised_images(amount_images); // mock output
12   double start = omp_get_wtime();
13
14   // #pragma omp parallel for is an "OpenMP shortcut"
15 #pragma omp parallel for num_threads(4) // denoise a bunch of images
16   for (int i = amount_images - 1; i > -1; --i) {
17     denoised_images[i] = denoise_image(images[i], i * i);
18   }
19
20   cout << omp_get_wtime() - start << " seconds" << endl;
21 }
```

The Schedule Clause

The schedule clause affects **how loop iterations are mapped onto threads when using** `#pragma omp for`

By far the **most important schedule clauses** are:

- **static** (we make the scheduling decision at compile time)
- **dynamic** (deciding at runtime, how to break up work between the threads)

Usage:

- `schedule(static [,chunk])`
  Deal-out blocks of iterations of size "chunk" to each thread.
- `schedule(dynamic [,chunk])`
  Each thread grabs "chunk" iterations off a queue until all iterations have been handled.

Schedule Clause Demo

schedule_clause.cpp

```
1  int main() {
2  #pragma omp parallel num_threads(4)
3    {
4      // each thread has its own stringstream instance
5      stringstream thread_iterations;
6      // task for you: experiment with different schedules
7  #pragma omp for schedule(static, 8)
8      for (int i = 0; i < 32; ++i) {
9        // write the iterations of a thread to the stream
10       thread_iterations << i << " ";
11     }
12     // only one thread is allowed to output to the console
13 #pragma omp critical
14       cout << "thread " << omp_get_thread_num()
15            << " computed the iterations: "
16            << thread_iterations.str() << endl;
17   }
18 }
```

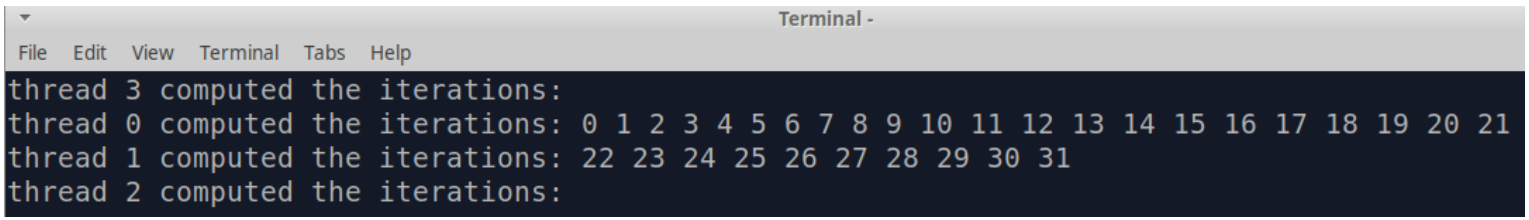
Choosing the Right Schedule Clause

If each iteration in the for loop needs the **same workload**, make the **schedule static**. (not necessarily true if you have efficiency cores)

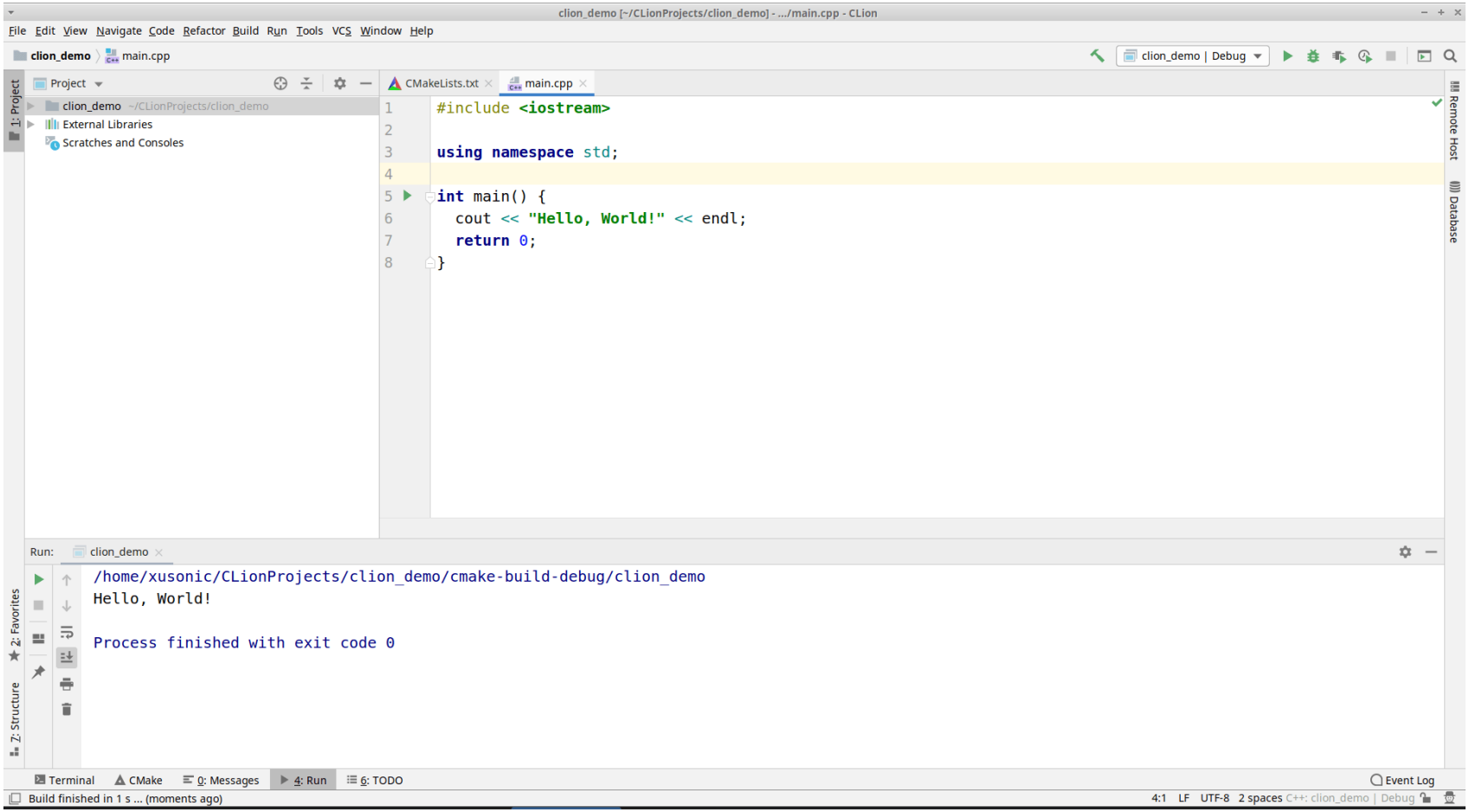
If the **workload** is **unbalanced** between iterations, make the **schedule dynamic**.

Coding Warmup

1. Fix the **race condition bug** on page 13 with a **`std::mutex`**.
2. **Reduce** the **runtime** of the image denoising program on slide 16 by adding an **appropriate schedule**.
3. What **schedule** on slide 18 produces the **following (bad) pattern**?



Clion Demo


clion_demo [~/CLionProjects/clion_demo] - .../main.cpp - CLion
File Edit View Navigate Code Refactor Build Run Tools VCS Window Help
clion_demo main.cpp
clion_demo | Debug
Project
clion_demo ~/CLionProjects/clion_demo
External Libraries
Scratches and Consoles
CMakeLists.txt
main.cpp
#include <iostream>
using namespace std;
int main() {
cout << "Hello, World!" << endl;
return 0;
}
Run: clion_demo
/home/xusonic/CLionProjects/clion_demo/cmake-build-debug/clion_demo
Hello, World!
Process finished with exit code 0
Terminal CMake 0: Messages 4: Run 6: TODO Event Log
Build finished in 1 s ... (moments ago)
4:1 LF UTF-8 2 spaces C++: clion_demo | Debug
1: Project
2: Favorites
7: Structure
Remote Host
Database

Exam Assignments

- What causes **false sharing**? Describe it visually by creating your own graphic.
- Read the paper **There's plenty of room at the Top: What will drive computer performance after Moore's law?**

  Click here to download the paper
  - Explain in detail the figure ***Performance gains after Moore's law ends.*** (on the first page)
- Do the **coding warmup** on **slide 20**. (optional)
- **Rewrite** the program for **estimating** π from the **last lecture** (`pi_numerical_integration.cpp` 📌) so that it is parallelized with the `#pragma omp for` construct.